

B1B13PPS

Průmyslové počítačové systémy

Michal Brejcha

`brejcmic@fel.cvut.cz`

ČVUT v Praze, FEL

Praha, 2025

Obsah

- 1 Adresace paměti
- 2 Zásobník - paměť STACK
- 3 Program Pole - indexové adresování
- 4 Program Souběh - proměnné v RAM

Téma

- 1 Adresace paměti
- 2 Zásobník - paměť STACK
- 3 Program Pole - indexové adresování
- 4 Program Souběh - proměnné v RAM

Adresace paměti instrukcí

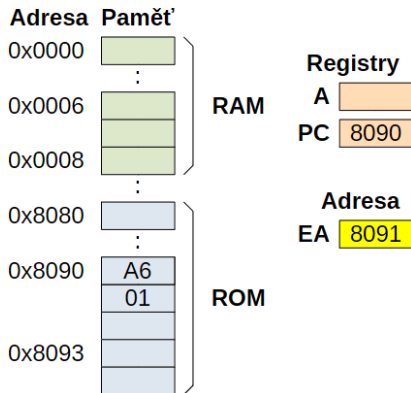
- **Každá instrukce určitým způsobem pracuje s pamětí:**
 - 1 načtení hodnoty z paměti,
 - 2 provedení operace s načtenou hodnotou,
 - 3 uložení výsledku zpět do paměti.
- **Zdrojovou nebo cílovou pamětí může být v případě procesorů STM8 jakákoliv paměť v jednotném adresním prostoru:**
 - RAM,
 - ROM,
 - zásobník (STACK) - zásobníkové adresování, ...
- **Existuje více možností jak instrukce předává adresu paměti:**
 - implicitní (inherentní) adresování - instrukce bez operandů,
 - bezprostřední adresování,
 - přímé adresování,
 - indexové adresování,
 - nepřímé adresování,
 - relativní adresování - skoky.

Implicitní adresování

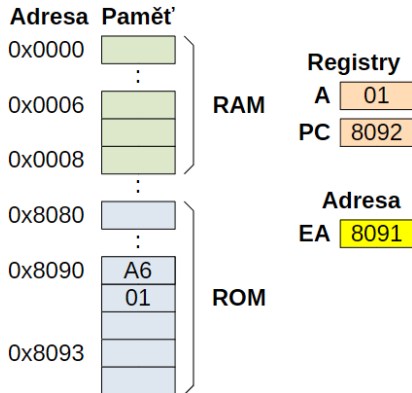
- Instrukce nevykonává operaci s pamětí - nemá operandy,
- strojový kód plně specifikuje vykonávanou operaci:
 - **NOP** - provedení prázdné (žádné) operace,
 - **RET** - návrat z podprogramu (přestože se čte z vrcholu zásobníku),
 - **MUL, DIV** - násobení a dělení, (pracuje jen s registry **A** a **X** nebo **Y**),
 - **SCF** - nastavení příznaku **CARRY**, atd.
- strojový kód instrukce má 1 nebo 2 byte.

Schéma výpočtu adresy paměti

Před výkonem instrukce



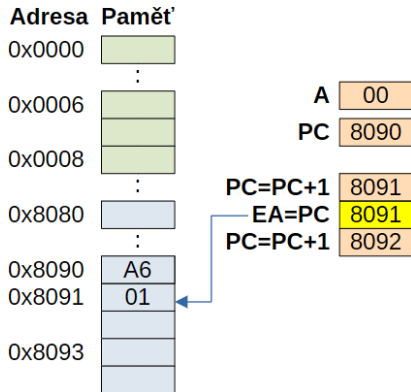
Po vykonání instrukce



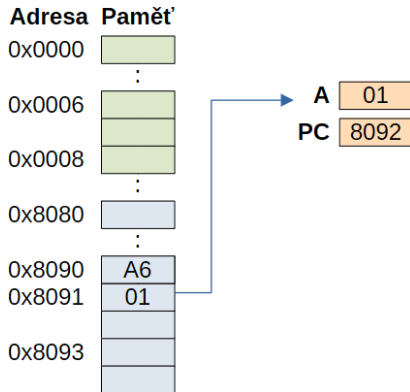
Bezprostřední adresace

Příklad: LD A, #\$01 ; křížek před hodnotou

Před výkonem instrukce



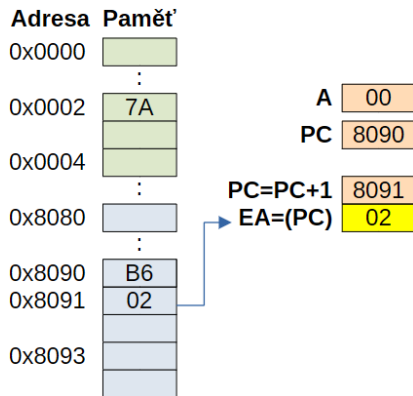
Po vykonání instrukce



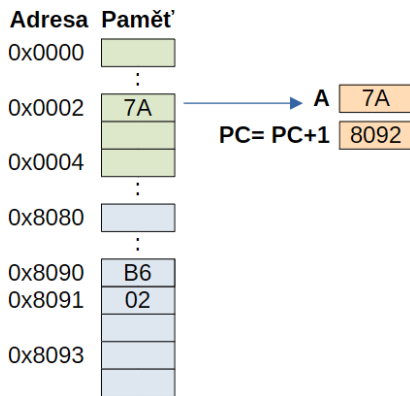
Přímá adresace - krátká adresa

Příklad: LD A, \$02 ; adresa má 1 byte

Před výkonem instrukce



Po vykonání instrukce

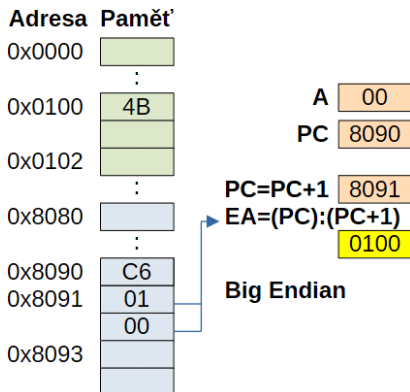


Adresace jen v rozsahu 0-255 (0x00 až 0xFF).

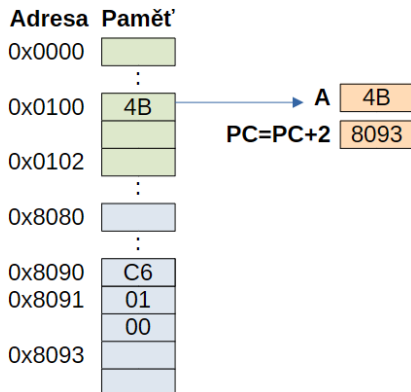
Přímá adresace - dlouhá adresa

Příklad: LD A, \$100 ; adresa má 2 byte

Před výkonem instrukce



Po vykonání instrukce



Adresace jen v rozsahu 0x0000 až 0xFFFF.

Přímá adresace

- Pro instrukce **CALLF** a **JPF** (skoky) se používá ještě rozšířená dlouhá adresace skládající se ze 3 byte:
 - uložení adresy v paměti v **Big Endian**,
 - po provedení instrukce se Program Counter zvýší o 3.
- Přímá adresace je používána většinou instrukcí:
 - čtení z paměti a zapisování do paměti **LD**, **MOV**,
 - logické operace **OR**, **XOR**, ...
 - aritmetické operace **ADD**, **SUB**, ...
 - bitové rotace a posuny **RLC**, **SLL**, ...
 - také ji používají absolutní skoky **CALL**, **JP**,
 - a operace ukládání a čtení zásobníku **PUSH**, **POP**.

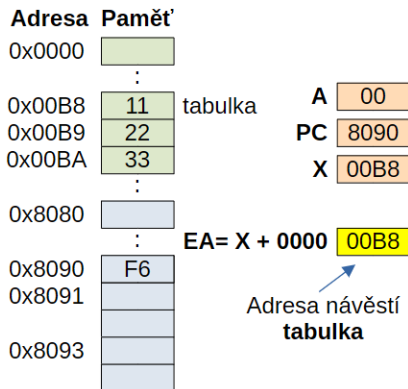
Indexová adresace

- Používáme pro adresaci polí, tabulek apod.
- má dvě části:
 - **bázi**, která udává počátek adresy a
 - **posunutí** nebo také **offset**, který udává posun (počet kroků, byte) od báze adresy,
- rozlišujeme:
 - indexové s nulovým offsetem = nepřímá registrová adresace,
 - indexové s **ne**nulovým offsetem = indexová adresace,
- pro indexovou adresaci (tj. zápis offsetu) se používají indexové registry **X**, **Y**.

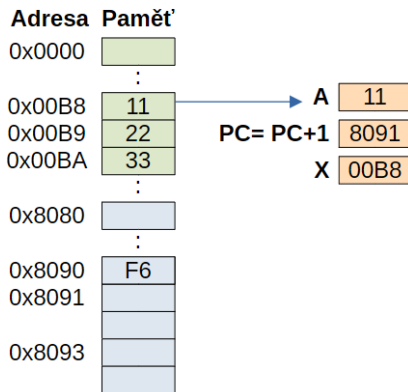
Indexová adresace s nulovým offsetem

Příklad: LD A, (X) ; báze = 0

Před výkonem instrukce



Po vykonání instrukce

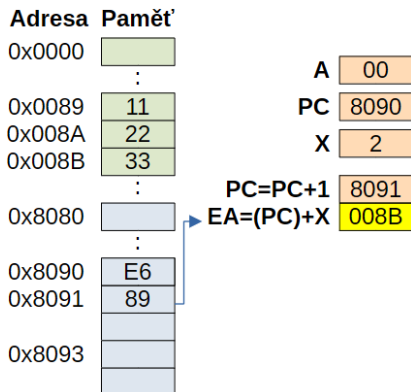


Dovoleno adresovat jen prostor 0x00 až 0xFF.

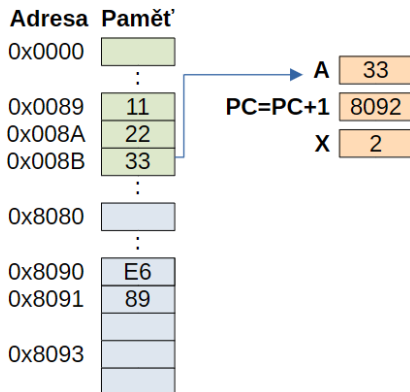
Indexová adresace - krátká adresa

Příklad: LD A, (\$89, X) ; báze = 0x89, posun 2

Před výkonem instrukce



Po vykonání instrukce

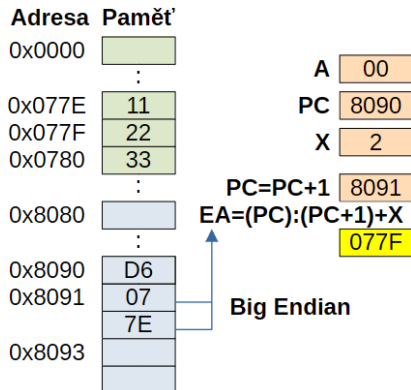


Dovoleno adresovat jen prostor 0x00 až 0x1FE.

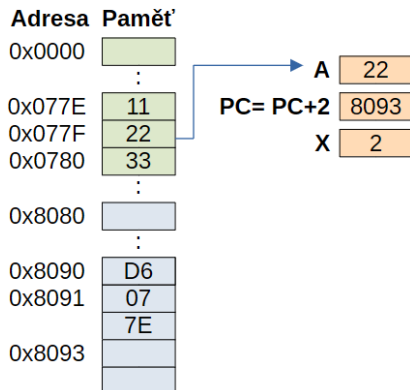
Indexová adresace - dlouhá adresa

Příklad: LD A, (\$077E, X) ; báze = 0x077E, posun 1

Před výkonem instrukce



Po vykonání instrukce



Lze adresovat celý prostor 128 kB.

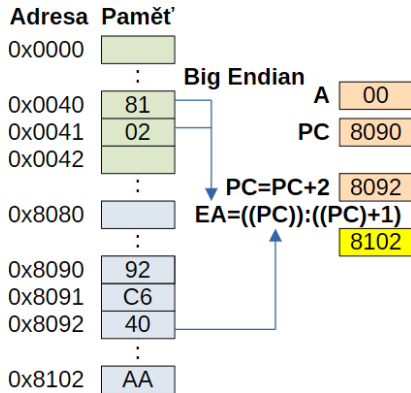
Nepřímá adresace

- **Adresa dat** je uložena pomocí ukazatele,
- **ukazatel** je jiné paměťové místo, které obsahuje cílovou adresu dat, tj.
 - 1 na adrese ukazatele je přečtena hodnota,
 - 2 tato hodnota je použita jako adresa pro čtení dat.
- opět existují varianty:
 - nepřímé adresování s krátkým nebo dlouhým ukazatelem:
 - **LD** A, [\$40.w]
 - **LD** A, [\$1040.w]
 - nepřímé adresování indexové s krátkým nebo dlouhým ukazatelem:
 - **LD** A, ([\$40.w], X)
 - **LD** A, ([\$1040.w], X)
- **z „ukazované“ pozice čteme dlouhou adresu ⇒ adresujeme celý prostor 64 kB.**

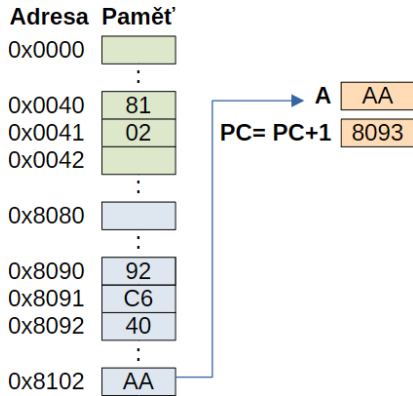
Nepřímá adresace - krátký ukazatel

Příklad: LD A, [\$40.w] ; ukazatel ma adresu 0x0040

Před výkonem instrukce



Po vykonání instrukce



Relativní adresace

- používá se především u podmíněných a relativních skoků:
 - **JRxx** navesti ; za xx se doplňuje reakce na příznak
 - **JRA** navesti ; relativní skok vždy
 - **CALLR** navesti ; relativní volání funkce
- pozor na absolutní skoky jako **JP**, ty používají přímé adresování,
- při relativní adresaci se vypočítává nová hodnota **PC** přičtením **8 bit znaménkového offsetu**,
- tedy lze skočit jen o 127 byte dále nebo o 128 byte zpět,
- relativní skoky zaberou méně cyklů než absolutní skoky (**JRA**(2 cy) proti **JP**(5 cy)).

Relativní adresace - relativní skok

Příklad: JREQ navesti ; skok pro $Z=1$ na navesti

Před výkonem instrukce

Adresa	Paměť	
0x0000		
	:	
0x0040		
0x0041		PC 8090
0x0042		
	:	
0x8080		PC=PC+1 8091
	:	TMP=(PC) 12
	:	PC=PC+1 8091
0x8090	27	Jen při skoku:
0x8091	12	EA=PC+TMP 80A3
0x8092		Jinak:
	:	EA=PC 8092
0x80A3		navesti

Skok pro $Z=1$ splněn

Adresa Paměť

0x0000	
⋮	
0x0040	
0x0041	
0x0042	
⋮	
0x8080	
⋮	
0x8090	27
0x8091	12
0x8092	
⋮	
0x80A3	

PC=EA 80A3

navesti ←

Téma

- 1 Adresace paměti
- 2 Zásobník - paměť STACK**
- 3 Program Pole - indexové adresování
- 4 Program Souběh - proměnné v RAM

Co je zásobník?

- speciální část paměti **RAM** fungující jako struktura **LIFO**,
- **Last In, First Out**: poslední zapsaná data jsou jako první vyčtena,
- na vrchol zásobníku ukazuje **Stack Pointer (SP = registr ukazatele)**,
- vrcholem se rozumí první neobsazené místo paměti,
- vrchol prázdného zásobníku je na nejvyšší adrese, plněním tak **SP** ukazuje stále na nižší adresy,

Adresa

0x1400

:

0x17FB

0x17FC

0x17FD

0x17FE

0x17FF

44

33
22
11

PUSH #\$44

SP

17FC

44
33
22
11

SP

17FB

A

44

33
22
11

POP A

SP

17FC

K čemu se zásobník používá?

- ukládají se do něj návratové adresy funkčních skoků:
 - **CALL**: do zásobníku uloží adresu následující instrukce a skočí na návěstí,
 - **RET**: ze zásobníku přečte adresu a skočí na ni.
- ukládají se do něj návratové adresy přerušení a záloha systémových registrů,
- ve funkcích do něj ukládáme (vytváříme) lokální proměnné.

Adresa

0x1400

:

0x17FB

0x17FC

0x17FD

0x17FE

0x17FF

44

33

22

11

SP

17FC

PUSH #\$44

44

33

22

11

SP

17FB

A

44

33

22

11

SP

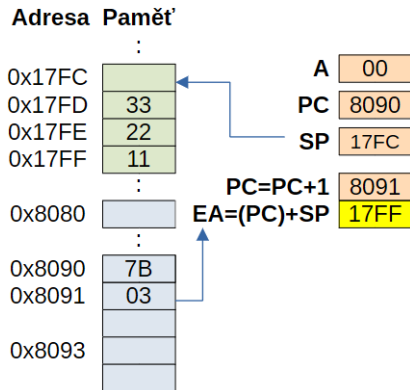
17FC

POP A

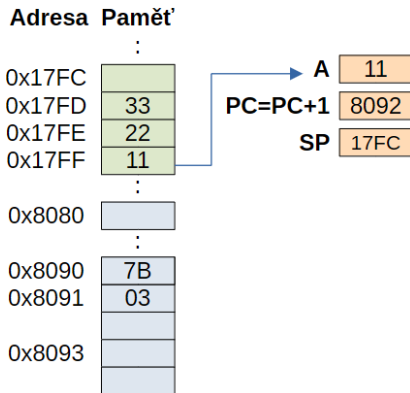
Indexová adresace zásobníku

Příklad: LD A, (\$03, SP) ; SP = první neobsazená adresa

Před výkonem instrukce



Po vykonání instrukce



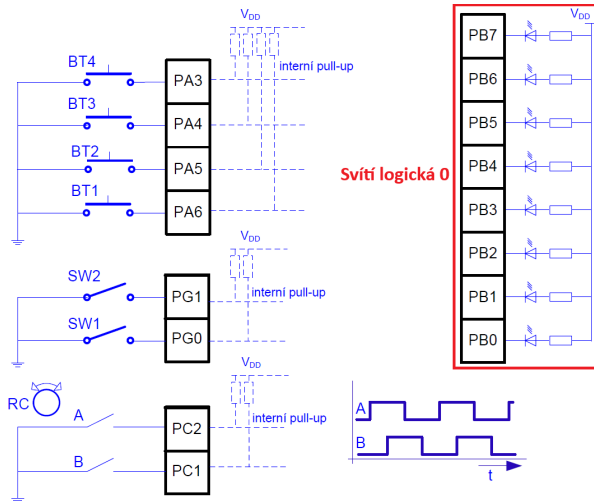
Téma

- 1 Adresace paměti
- 2 Zásobník - paměť STACK
- 3 Program Pole - indexové adresování**
- 4 Program Souběh - proměnné v RAM

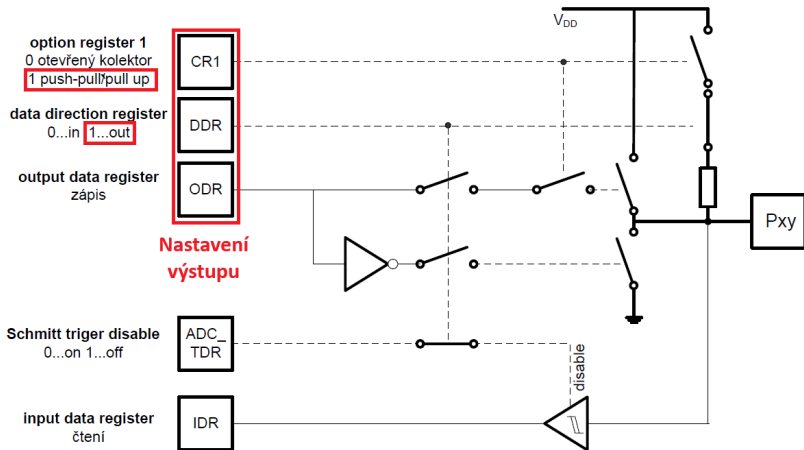
Zadání

- 1 Program spustí světelný efekt na LED přípravku,
- 2 LED budou rozsvěceny dle pevného zadání v paměti,
- 3 pro časování programu bude použita již napsaná funkce **wait**.

Výstupní port s LED u přípravku



Nastavení pinu procesoru



Program - definiční konstanty

```

9  ;          DEFINICNI KONSTANTY
10 ;-----
11          |          BYTES
12 ; konstanty pro nastaveni pinu portu B = LED
13 INI_PB_DDR      equ      %11111111      ; vse vystup
14 INI_PB_CR1      equ      %11111111      ; vse push/pull
15 INI_PB_ODR      equ      %11111111      ; 1 zhasne LED

```

- pojmenované hodnoty = konstanty,
- překladač nahradí název konstanty její hodnotou,
- pojmenování usnadňuje případný přepis konstanty, pokud se nachází na více místech v programu.

Program - hlavní smyčka

```

32 ;          PROGRAM
33 ;-----
34         segment 'rom'
35
36         ; inicializace vystupu LED
37 main.1  mov     PB_ODR, #INI_PB_ODR      ; stav vystupu pinu
38         mov     PB_CR1, #INI_PB_CR1     ; typ totomu pinu
39         mov     PB_DDR, #INI_PB_ODR     ; smer pinu
40
41         ; inicializace indexu
42 index   clr     X                        ; X = 0x0000
43
44         ; smyčka programu
45 smycka  ld      A, (pole,X)              ; A = pole[X]
46         cpl     A                        ; negace A
47         ld      PB_ODR, A                ; vystup LED = A
48         incw    X                        ; X = X+1 (16 bit)
49         call    wait                     ; cekat 0,5 s
50         cpw     X,#{poleKon-pole}        ; je konec pole?
51
52         jrult   smycka                    ; Slozene zavorky !!
53
54         jp      index                    ; skok na smycka
55         ; pri X < delka pole
56         ; skok na novou
57         ; inicializaci X

```

Inicializace

Smyčka

Program - definice tabulky

```
57          ; definice pole v pameti ROM
58          ; kazdy radek reprezentuje jeden stav LED
59 pole     dc.b      %00000000
60          dc.b      %10000000
61          dc.b      %01000000
62          dc.b      %00100000
63          dc.b      %00010000
64          dc.b      %00001000
65          dc.b      %00000100
66          dc.b      %00000010
67          dc.b      %00000001
68 poleKon  dc.b      %11111111 ; posledni radek se nepouzije
```

- pole je definováno v segmentu **rom**,
- direktiva **dc.b** přímo zapisuje hodnotu do paměti,
- návěští na konci je jen z důvodu snazšího určení délky tabulky (viz předchozí slide).

Téma

- 1 Adresace paměti
- 2 Zásobník - paměť STACK
- 3 Program Pole - indexové adresování
- 4 Program Souběh - proměnné v RAM**

Zadání

- 1 Program spustí světelný vypočtený efekt na LED přípravku,
- 2 vždy svítí dvě led o po stanoveném časovém kroku běží proti sobě,
- 3 program využije globální proměnné v RAM = pojmenovaný paměťový prostor,
- 4 pro časování programu bude použita již napsaná funkce **wait**.

Program - zakládání proměnných

```

24 ;          PROMENNE
25 ;-----
26         segment 'ram0'
27
28 ; promenna funkce wait v RAM
29 pocl     ds.b      1                ; rezervace 1 byte
30
31 ; globalni promenne pro posun vlevo a posun vpravo
32 levyP     ds.b      1                ; rezervace 1 byte
33 pravyP     ds.b      1                ; rezervace 1 byte

```

- Direktiva **ds.b** pouze přiřadí například symbolu **pravyP** hodnotu odpovídající první volné adrese v paměti RAM,
- lze vyhradit i více jak 1 byte, ale pak musíme k prostoru přistupovat jako k více bytové proměnné nebo jako k poli.

Program - hlavní smyčka

```

37 ;-----
38     segment 'rom'
39
40     ; inicializace vystupu LED
41 main.1 mov     PB_ODR, #INI_PB_ODR      ; stav vystupu pinu
42         mov     PB_CR1, #INI_PB_CR1    ; typ totemu pinu
43         mov     PB_DDR, #INI_PB_ODR    ; smer pinu
44
45     ; inicializace hodnot promennych
46 inihod mov     levyP, #%00000001      ; levyP = 00000001
47         mov     pravyP, #%10000000    ; pravyP = 10000000
48
49     ; smyčka programu
50 smycka ld      A, levyP                ; A = levyP
51         add     A, pravyP              ; A = A + pravyP
52         cpl     A                      ; negace A
53         ld      PB_ODR, A              ; vystup LED = A
54         call    wait                   ; cekat 0,5 s
55         sll     levyP                  ; bitovy pos. vlevo
56         srl     pravyP                 ; bitovy pos. vpravo
57
58         jreq    inihod                 ; skok na inihod
59         |      |                      ; pri pravyP = 0
60         jp      smycka                 ; skok na smycka

```

Následující přednáška

- Periferie procesoru STM8,
- ADC,
- čítač,
- PWM,
- sériová komunikace.

Děkuji za pozornost